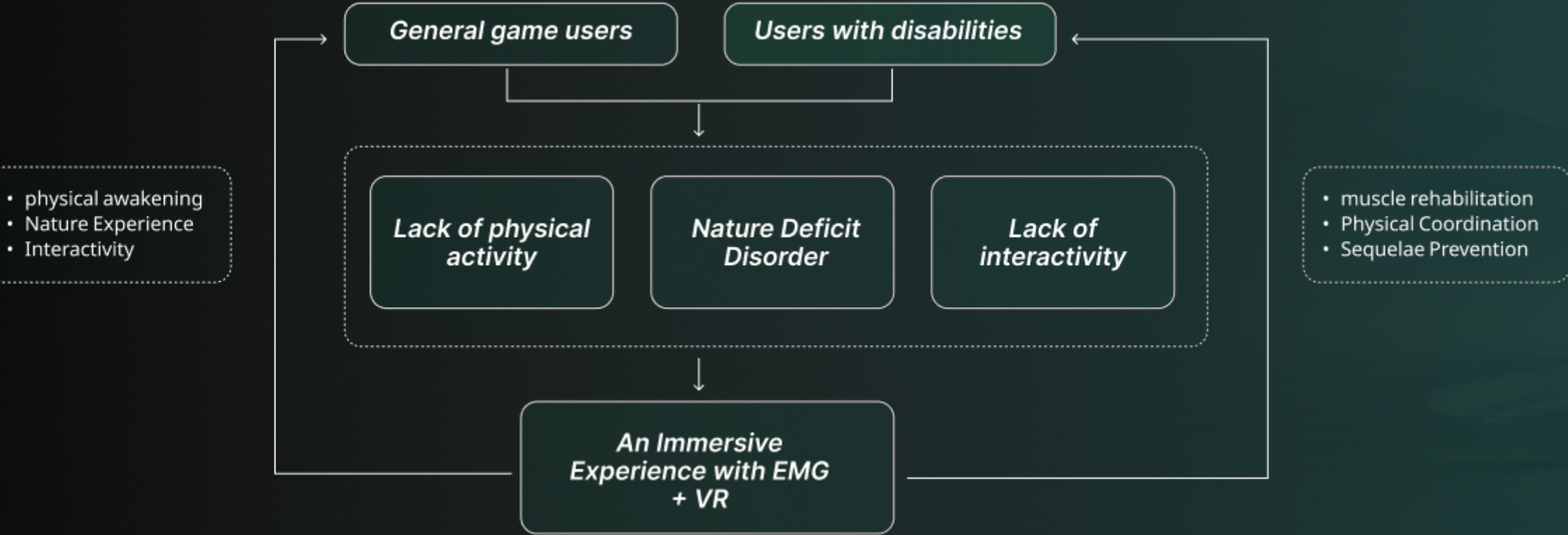


Primal Hunting - VR+EMG

Myoelectric Linkage Hunting Game Design

“Primal Hunting has created an immersive hunting game for sports enthusiasts and disabled users by innovatively combining VR technology and EMG sensing systems to provide both precise motor feedback and rehabilitation training, as well as a challenging entertainment experience.”



Vedio Link :
<https://youtu.be/pP1NrBxjgpU?si=FWaqbDQHkyG3yuSN>

Target Group Analysis

Primitive Hunting is an immersive hunting game that combines VR technology with EMG myoelectric sensing system, aiming to provide players with a realistic hunting experience and deep motion feedback. Through the intermingling of virtual and reality, players will explore and grow in diverse scenarios and experience the challenge and fun of hunting. This game is suitable for players who love sports, technology and competition.

Core User Profile

Population statistics

The core user group focuses on young males aged 18-35, with gaming and sports preferences highly in line with the project design direction.



Interests and Needs

Users are highly receptive to VR technology and expect novel motion feedback and immersion experience.



Potential consumer behavior

Health and social functions are important added values to attract potential users.

Gaming Habits



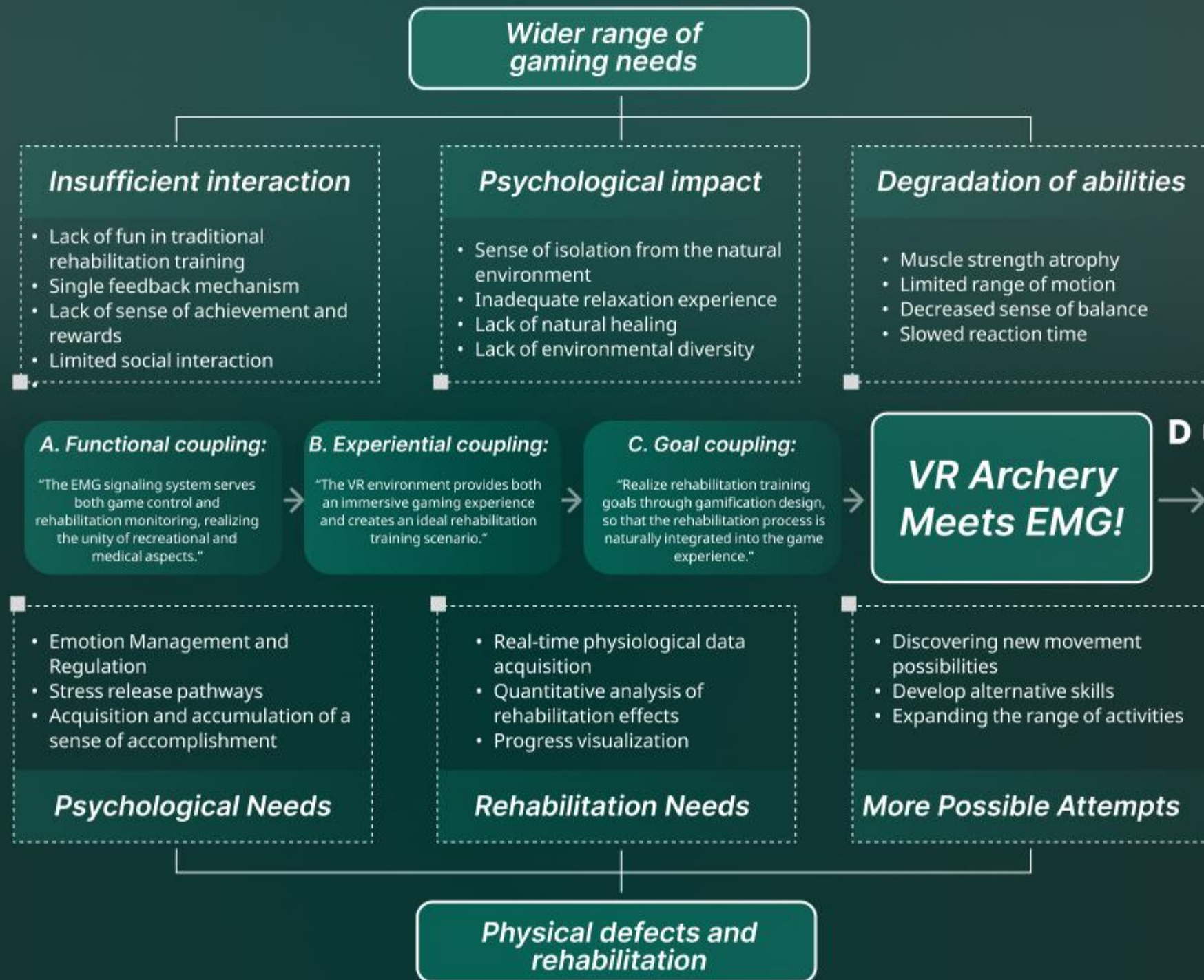
A high percentage of users have stable game consumption habits, with high commercialization potential.



User Needs Analysis

User needs of persons with disabilities

"Focus on the integrity and fun of the game experience, pursuing high-quality visual performance, smooth operational response, and rich social interaction features, while expecting challenging game mechanics and a clear progression system."



Common User Requirements

"Concerns about the accessibility and adaptability of the system require stable and reliable EMG control mechanisms, professional rehabilitation training guidance, and special interaction designs that take physical limitations into account, as well as a desire for rehabilitative outcomes and social acceptance through play."

Summary And Outlook

Primal Hunt fulfills just these multidimensional needs through an immersive VR experience with innovative EMG myoelectric sensing technology.

A Phantom Limb Pain Phenomenon

- 76% of upper limb amputees report experiencing phantom limb pain.
- Frequency and intensity of pain varies from person to person, but generally affects quality of life.

B Rehabilitation Difficulties

- Lack of timely feedback on progress
- Difficulty in controlling the intensity of training

C Restricted exercise

- Lack of targeted exercise instruction
- Fear of exercise leading to injury

D Problems with muscle atrophy

ZHANG Chao 34

"I've always wanted to try archery, but traditional equipment is difficult to maneuver."

WANG CHN

Adaptation period for EMG control for disabled users is about 2-3 days

EMG archery game effectively improves user's muscle control

XIAO Li 37

"I've always wanted to try archery, but traditional equipment is difficult to maneuver."

Creating high-quality virtual archery scenarios and interaction systems

Spatial localization and joystick tracking capabilities for VR devices

Arduino acquisition and processing of EMG signals

REASONS FOR CHOOSING A HUNTING THEME

PERCEPTUAL CREATIVITY

Market factors

- Theme uniqueness: Hunting theme is rare and attractive.
- Sport fit: perfect combination with EMG technology to expand the market.
- User Stickiness: Long growth cycle of skills to promote user retention.

● Theme uniqueness

- Hunting theme is relatively rare in the game market, which can attract user groups who like fresh and challenging content, and has obvious competitive differentiation advantages.

● High user stickiness

- The hunting theme is naturally characterized by growth and challenges, and players need to continuously hone their skills and unlock equipment, which results in long-term game motivation and a high retention rate.

● Technological fit

- Hunting activities require precise movement and power control, and the combination of EMG technology and VR immersion experience can realize highly realistic interaction effects, highlighting the technological advantages.

● Combination of sports and entertainment

- Hunting activities integrate physical strength and strategic needs, which not only satisfy players' sports needs, but also provide an intense and exciting entertainment experience, which is in line with the trend of designing both health and fun.

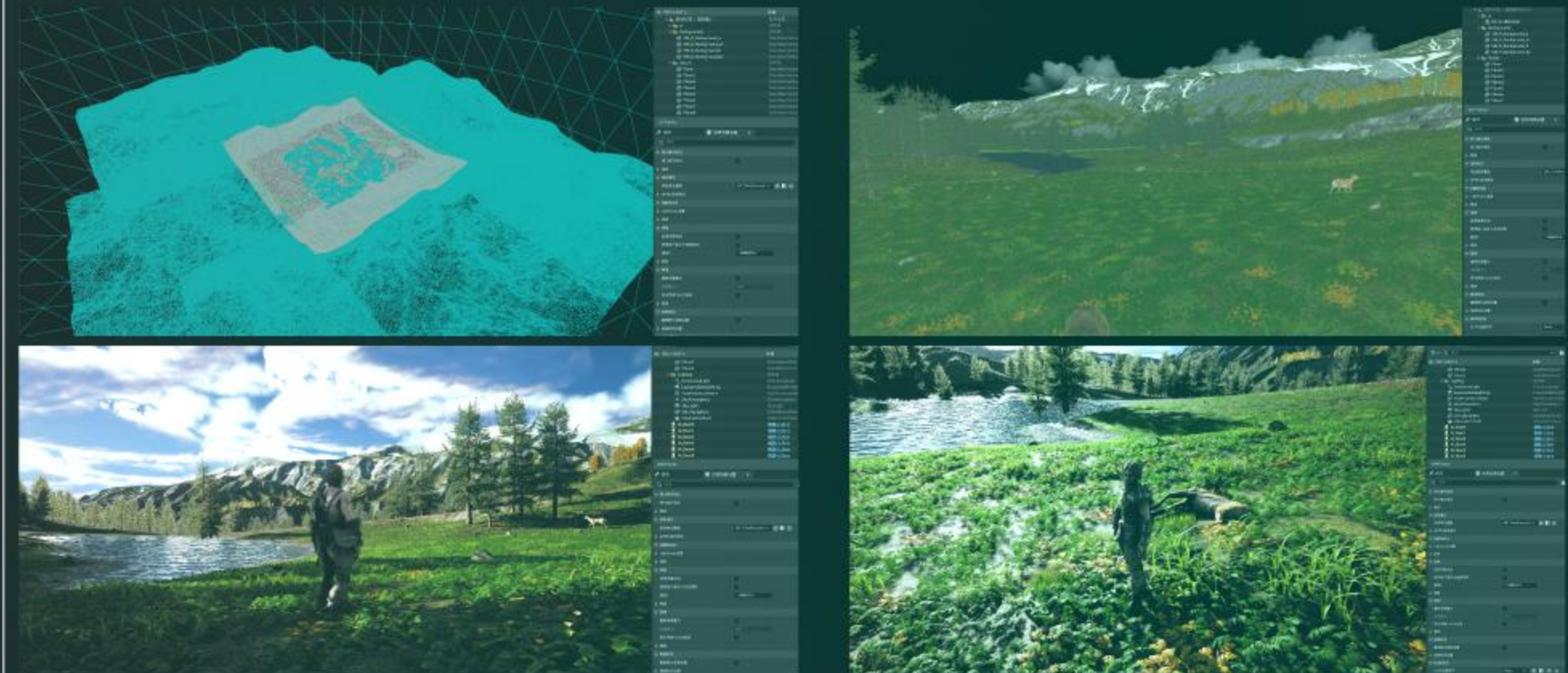
Game Design Advantages

Vertical Correlation

- Rich Action Design: The game features diverse playstyles like archery, spear throwing, and traps, each with unique mechanics that encourage strategy and interaction.
- Realistic Force Feedback: Advanced haptic technology captures muscle control, providing realistic feedback for actions like drawing a bow, throwing a spear, or setting traps, enhancing immersion.
- Skill Growth Depth: A multi-level skill tree lets players specialize and improve in areas like archery, spear throwing, and traps, offering new techniques and deeper gameplay as they progress.

In a futuristic and ancient world, the player will take on the role of a hunter, recreating the instincts and challenges of primitive hunting through high-tech means. In the treacherous wilderness, players will have to continue practicing, mastering their hunting skills and surviving the laws of nature.

Scenario Design



Primitive Forest

- Primitive Forest: Towering trees, gurgling streams, and hidden caves create a picture of a vibrant rainforest. Players will have to face complex terrain and changing weather, and the pursuit of exploration and prey will be a test of endurance and strategy.
- Tundra Grassland: The vast plains and low shrubs are a beautiful contrast, and the breeze stirs up waves of grass, mesmerizing you. However, the open terrain also makes the competition between hunter and prey more tense and exciting, and every step may be the key to success or failure.



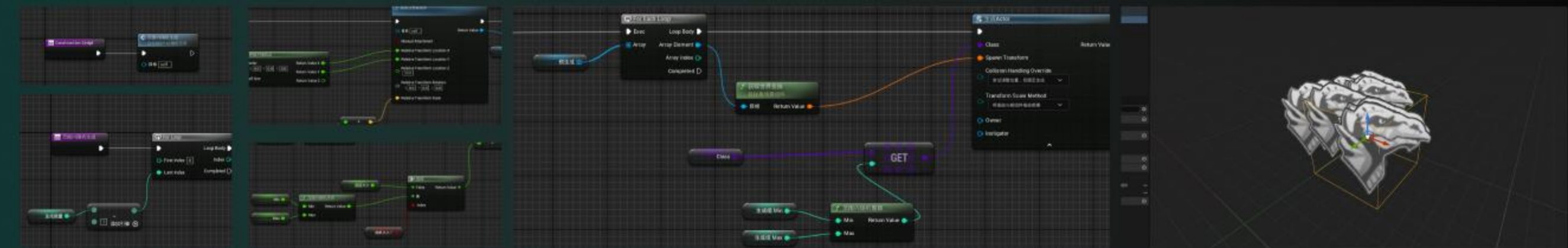
Tundra Steppe

The scene design of Primitive Hunting focuses on creating realistic natural environments with rich interactive details, bringing players an immersive and immersive experience. Each scene has been carefully polished to show the magnificence and variety of nature, but also combines dynamic interaction technology to add a sense of hierarchy and realism to the hunting challenge.

FUNCTIONAL REALIZATION

Random AI generation

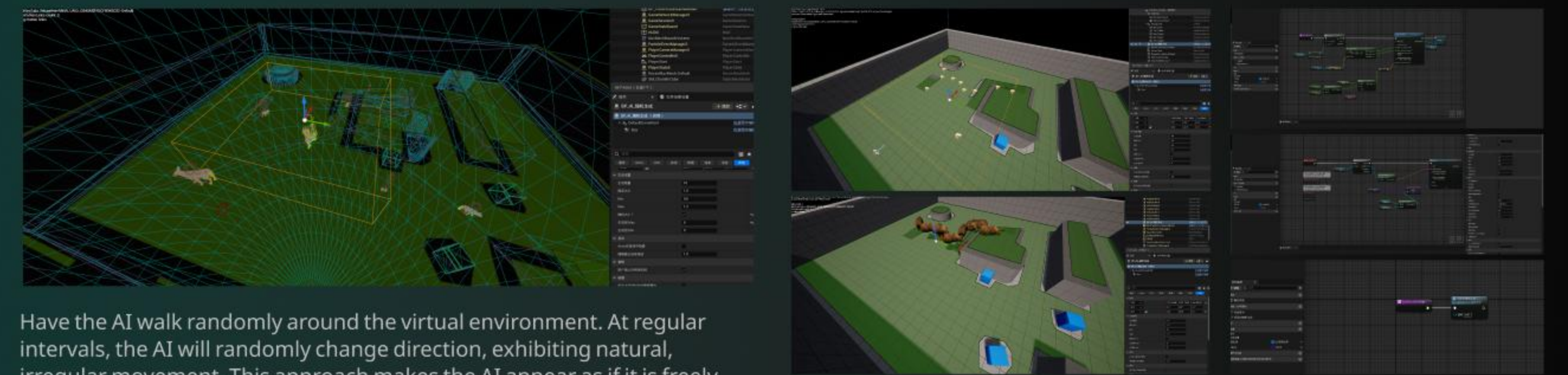
Drawing inspiration from Mondrian's geometric abstract art, we visualize rules and data as a growing geometric structure system. Through UE's procedural nodes, we create a dynamically evolving data landscape that demonstrates how rules gradually consume human life. Each stage symbolizes different dimensions of data-driven judgment, ultimately forming a vast "data mountain range."



Dynamic Behavior Mode

- The AI character has a diverse behavioral tree that adjusts its behavior in real time according to the player's actions, changes in the environment, and its own state. For example, the prey will choose to run, hide, or fight back when it senses the player is approaching, creating a dynamic interactive experience.
- The technology uses multi-threaded processing of AI logic to ensure that when multiple characters are active at the same time, they can still maintain smooth operation and timely response, providing technical support for large-scale hunting scenarios.

Dynamic Behavioral Model



Have the AI walk randomly around the virtual environment. At regular intervals, the AI will randomly change direction, exhibiting natural, irregular movement. This approach makes the AI appear as if it is freely exploring the environment.

EMG INTERACTION DESIGN

This project integrates EMG (Electromyography) sensing technology with VR archery, creating an adaptive interaction system that serves both regular users and those with disabilities. The system utilizes dual EMG sensors strategically placed at the thenar muscle (thumb base) and forearm flexors to capture precise muscle activation patterns.

Sensor fabrication process and circuits



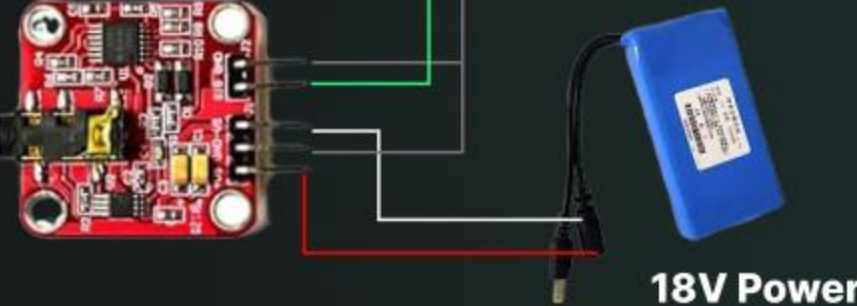
Adaptation Recommendations:

Standard users:
Base Threshold: 300-400
Linear mapping strength
Standard filter parameter

Strength Users:
Raised Threshold: 400-500
Expand signal range
Increase sensitivity

Precision users:
Decrease threshold: 250-350
Increase smoothing
Increase stability requirements

EMG



18V Power

Arm Amputation User



Primary: biceps/triceps group on stump
Secondary: anterior/posterior deltoid
Control: draw/release controlled by strength of muscle contraction

Arduino IDE Code

```
1 #include <SoftwareSerial.h>
2 #define BLUETOOTH_RX 2
3 #define BLUETOOTH_TX 4
4 #define BLUETOOTH_SPEED 9600
5 #define BLUETOOTH_SPEED 9600
6 #define BLUETOOTH_SPEED 9600
7 #define BLUETOOTH_SPEED 9600
8 #define BLUETOOTH_SPEED 9600
9 #define BLUETOOTH_SPEED 9600
10 #define BLUETOOTH_SPEED 9600
11 #define BLUETOOTH_SPEED 9600
12 #define BLUETOOTH_SPEED 9600
13 #define BLUETOOTH_SPEED 9600
14 #define BLUETOOTH_SPEED 9600
15 #define BLUETOOTH_SPEED 9600
16 #define BLUETOOTH_SPEED 9600
17 #define BLUETOOTH_SPEED 9600
18 #define BLUETOOTH_SPEED 9600
19 #define BLUETOOTH_SPEED 9600
20 #define BLUETOOTH_SPEED 9600
21 #define BLUETOOTH_SPEED 9600
22 #define BLUETOOTH_SPEED 9600
23 #define BLUETOOTH_SPEED 9600
24 #define BLUETOOTH_SPEED 9600
25 #define BLUETOOTH_SPEED 9600
26 #define BLUETOOTH_SPEED 9600
27 #define BLUETOOTH_SPEED 9600
28 #define BLUETOOTH_SPEED 9600
29 #define BLUETOOTH_SPEED 9600
30 #define BLUETOOTH_SPEED 9600
31 #define BLUETOOTH_SPEED 9600
32 #define BLUETOOTH_SPEED 9600
33 #define BLUETOOTH_SPEED 9600
34 #define BLUETOOTH_SPEED 9600
35 #define BLUETOOTH_SPEED 9600
36 #define BLUETOOTH_SPEED 9600
37 #define BLUETOOTH_SPEED 9600
38 #define BLUETOOTH_SPEED 9600
39 #define BLUETOOTH_SPEED 9600
40 #define BLUETOOTH_SPEED 9600
41 #define BLUETOOTH_SPEED 9600
42 #define BLUETOOTH_SPEED 9600
43 #define BLUETOOTH_SPEED 9600
44 #define BLUETOOTH_SPEED 9600
45 #define BLUETOOTH_SPEED 9600
46 #define BLUETOOTH_SPEED 9600
47 #define BLUETOOTH_SPEED 9600
48 #define BLUETOOTH_SPEED 9600
49 #define BLUETOOTH_SPEED 9600
50 #define BLUETOOTH_SPEED 9600
51 #define BLUETOOTH_SPEED 9600
52 #define BLUETOOTH_SPEED 9600
53 #define BLUETOOTH_SPEED 9600
54 #define BLUETOOTH_SPEED 9600
55 #define BLUETOOTH_SPEED 9600
56 #define BLUETOOTH_SPEED 9600
57 #define BLUETOOTH_SPEED 9600
58 #define BLUETOOTH_SPEED 9600
59 #define BLUETOOTH_SPEED 9600
60 #define BLUETOOTH_SPEED 9600
61 #define BLUETOOTH_SPEED 9600
62 #define BLUETOOTH_SPEED 9600
63 #define BLUETOOTH_SPEED 9600
64 #define BLUETOOTH_SPEED 9600
65 #define BLUETOOTH_SPEED 9600
66 #define BLUETOOTH_SPEED 9600
67 #define BLUETOOTH_SPEED 9600
68 #define BLUETOOTH_SPEED 9600
69 #define BLUETOOTH_SPEED 9600
70 #define BLUETOOTH_SPEED 9600
71 #define BLUETOOTH_SPEED 9600
72 #define BLUETOOTH_SPEED 9600
73 #define BLUETOOTH_SPEED 9600
74 #define BLUETOOTH_SPEED 9600
75 #define BLUETOOTH_SPEED 9600
76 #define BLUETOOTH_SPEED 9600
77 #define BLUETOOTH_SPEED 9600
78 #define BLUETOOTH_SPEED 9600
79 #define BLUETOOTH_SPEED 9600
80 #define BLUETOOTH_SPEED 9600
81 #define BLUETOOTH_SPEED 9600
82 #define BLUETOOTH_SPEED 9600
83 #define BLUETOOTH_SPEED 9600
84 #define BLUETOOTH_SPEED 9600
85 #define BLUETOOTH_SPEED 9600
86 #define BLUETOOTH_SPEED 9600
87 #define BLUETOOTH_SPEED 9600
88 #define BLUETOOTH_SPEED 9600
89 #define BLUETOOTH_SPEED 9600
90 #define BLUETOOTH_SPEED 9600
91 #define BLUETOOTH_SPEED 9600
92 #define BLUETOOTH_SPEED 9600
93 #define BLUETOOTH_SPEED 9600
94 #define BLUETOOTH_SPEED 9600
95 #define BLUETOOTH_SPEED 9600
96 #define BLUETOOTH_SPEED 9600
97 #define BLUETOOTH_SPEED 9600
98 #define BLUETOOTH_SPEED 9600
99 #define BLUETOOTH_SPEED 9600
100 #define BLUETOOTH_SPEED 9600
```

Receive Bluetooth data and forward to UE5

```
1 #include <Arduino.h>
2 #include <SoftwareSerial.h>
3 #include <Keyboard.h>
4 #include <EEPROM.h>
5 #include <Wire.h>
6 #include <Adafruit_GFX.h>
7 #include <Adafruit_SSD1306.h>
8 #include <Adafruit_NeoPixel.h>
9 #include <Adafruit_NeoPixel.h>
10 #include <Adafruit_NeoPixel.h>
11 #include <Adafruit_NeoPixel.h>
12 #include <Adafruit_NeoPixel.h>
13 #include <Adafruit_NeoPixel.h>
14 #include <Adafruit_NeoPixel.h>
15 #include <Adafruit_NeoPixel.h>
16 #include <Adafruit_NeoPixel.h>
17 #include <Adafruit_NeoPixel.h>
18 #include <Adafruit_NeoPixel.h>
19 #include <Adafruit_NeoPixel.h>
20 #include <Adafruit_NeoPixel.h>
21 #include <Adafruit_NeoPixel.h>
22 #include <Adafruit_NeoPixel.h>
23 #include <Adafruit_NeoPixel.h>
24 #include <Adafruit_NeoPixel.h>
25 #include <Adafruit_NeoPixel.h>
26 #include <Adafruit_NeoPixel.h>
27 #include <Adafruit_NeoPixel.h>
28 #include <Adafruit_NeoPixel.h>
29 #include <Adafruit_NeoPixel.h>
30 #include <Adafruit_NeoPixel.h>
31 #include <Adafruit_NeoPixel.h>
32 #include <Adafruit_NeoPixel.h>
33 #include <Adafruit_NeoPixel.h>
34 #include <Adafruit_NeoPixel.h>
35 #include <Adafruit_NeoPixel.h>
36 #include <Adafruit_NeoPixel.h>
37 #include <Adafruit_NeoPixel.h>
38 #include <Adafruit_NeoPixel.h>
39 #include <Adafruit_NeoPixel.h>
40 #include <Adafruit_NeoPixel.h>
41 #include <Adafruit_NeoPixel.h>
42 #include <Adafruit_NeoPixel.h>
43 #include <Adafruit_NeoPixel.h>
44 #include <Adafruit_NeoPixel.h>
45 #include <Adafruit_NeoPixel.h>
46 #include <Adafruit_NeoPixel.h>
47 #include <Adafruit_NeoPixel.h>
48 #include <Adafruit_NeoPixel.h>
49 #include <Adafruit_NeoPixel.h>
50 #include <Adafruit_NeoPixel.h>
51 #include <Adafruit_NeoPixel.h>
52 #include <Adafruit_NeoPixel.h>
53 #include <Adafruit_NeoPixel.h>
54 #include <Adafruit_NeoPixel.h>
55 #include <Adafruit_NeoPixel.h>
56 #include <Adafruit_NeoPixel.h>
57 #include <Adafruit_NeoPixel.h>
58 #include <Adafruit_NeoPixel.h>
59 #include <Adafruit_NeoPixel.h>
60 #include <Adafruit_NeoPixel.h>
61 #include <Adafruit_NeoPixel.h>
62 #include <Adafruit_NeoPixel.h>
63 #include <Adafruit_NeoPixel.h>
64 #include <Adafruit_NeoPixel.h>
65 #include <Adafruit_NeoPixel.h>
66 #include <Adafruit_NeoPixel.h>
67 #include <Adafruit_NeoPixel.h>
68 #include <Adafruit_NeoPixel.h>
69 #include <Adafruit_NeoPixel.h>
70 #include <Adafruit_NeoPixel.h>
71 #include <Adafruit_NeoPixel.h>
72 #include <Adafruit_NeoPixel.h>
73 #include <Adafruit_NeoPixel.h>
74 #include <Adafruit_NeoPixel.h>
75 #include <Adafruit_NeoPixel.h>
76 #include <Adafruit_NeoPixel.h>
77 #include <Adafruit_NeoPixel.h>
78 #include <Adafruit_NeoPixel.h>
79 #include <Adafruit_NeoPixel.h>
80 #include <Adafruit_NeoPixel.h>
81 #include <Adafruit_NeoPixel.h>
82 #include <Adafruit_NeoPixel.h>
83 #include <Adafruit_NeoPixel.h>
84 #include <Adafruit_NeoPixel.h>
85 #include <Adafruit_NeoPixel.h>
86 #include <Adafruit_NeoPixel.h>
87 #include <Adafruit_NeoPixel.h>
88 #include <Adafruit_NeoPixel.h>
89 #include <Adafruit_NeoPixel.h>
90 #include <Adafruit_NeoPixel.h>
91 #include <Adafruit_NeoPixel.h>
92 #include <Adafruit_NeoPixel.h>
93 #include <Adafruit_NeoPixel.h>
94 #include <Adafruit_NeoPixel.h>
95 #include <Adafruit_NeoPixel.h>
96 #include <Adafruit_NeoPixel.h>
97 #include <Adafruit_NeoPixel.h>
98 #include <Adafruit_NeoPixel.h>
99 #include <Adafruit_NeoPixel.h>
100 #include <Adafruit_NeoPixel.h>
```

Receive EMG data from Arduino via serial port
Convert EMG signals to keyboard events:

Press 'E' key when EMG1 exceeds threshold (draw bow)
Release 'E' key and hit space bar when EMG2 exceeds threshold (release bow)

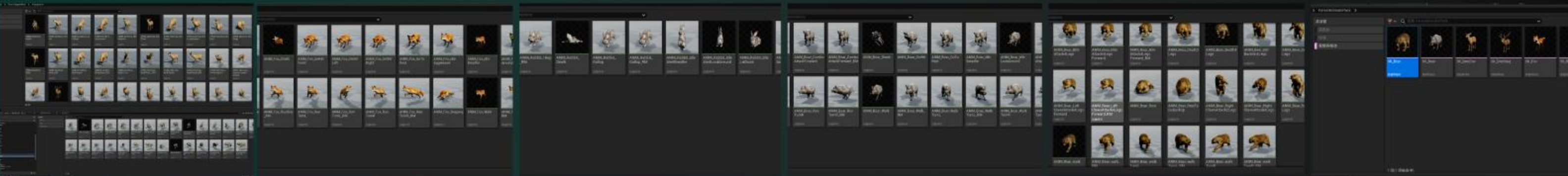
Upper EMG view



Due to the responsibility of the muscle electricity at present can only be established in the serial port simple signals for archery angle and position determination still can not be carried out.

MATERIAL SHOWCASE

During the project, I used a number of models and animation materials to enhance the overall experience. With these materials, I was able to quickly build beautifully crafted characters, props and environments to ensure a visually rich and realistic game. At the same time, the animation materials provided a smooth representation of the character and object movements, adding a sense of dynamism and interactivity to the game.

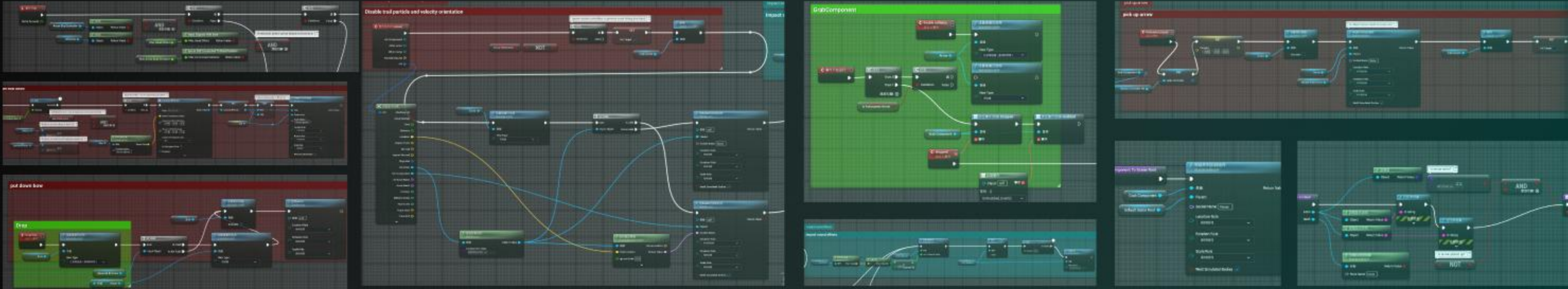


RENDERING SHOW



VR Archery features

I implemented a set of VR interactive archery features in Unreal Engine using blueprints. Players can simulate a real bow drawing motion, aim and fire arrows through VR controllers. This system makes the process of archery more intuitive and immersive, providing a VR experience that is interactive and natural in its operation.



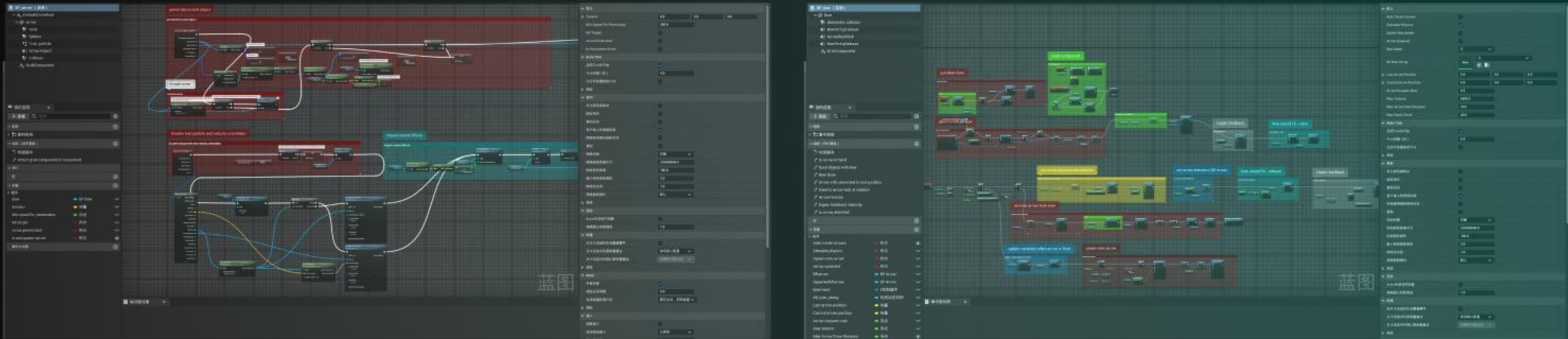
Bow control: Simulate the bow-drawing action through VR controllers to enhance interactivity.

Aiming Mechanism: Aim at the target naturally using the VR headset view.

Arrow Launch: Release the controller to launch the arrow, the flight trajectory is determined by the strength of drawing the bow.

Immersive experience: Realistic physical feedback enhances the immersion of the VR experience.

Replenishment



Dynamic target: During archery, the target can be static or dynamic, increasing the challenge and fun.

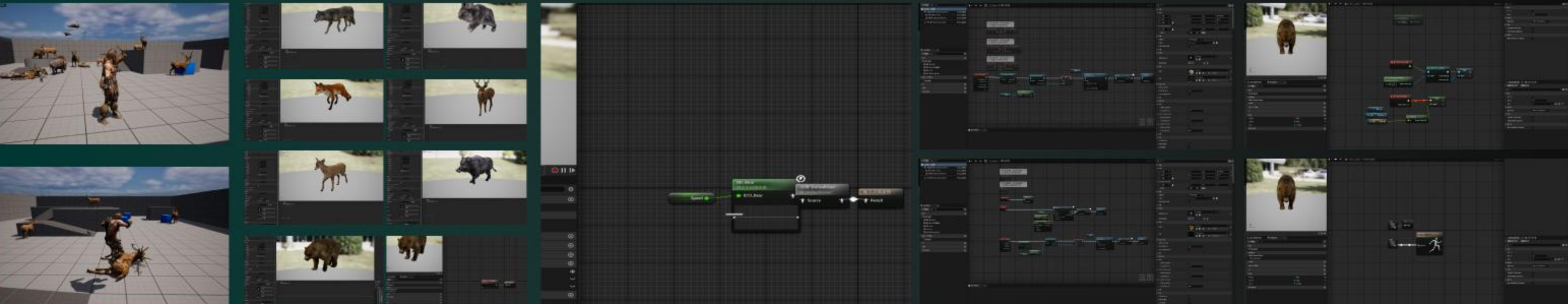
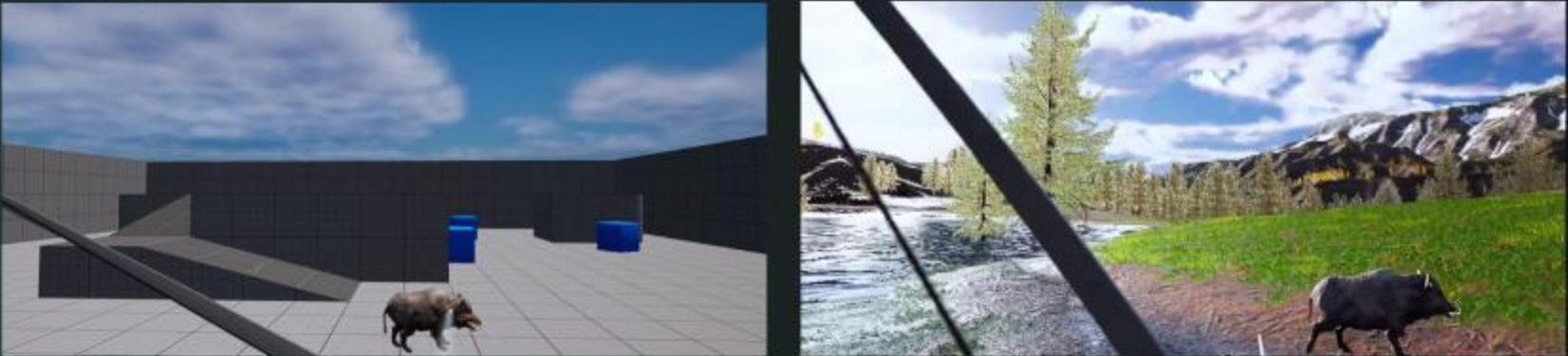
Interactive Feedback: Through the vibration and sound feedback of the VR handle, players can feel the power and accuracy of each shot, enhancing the sense of immersion.

Functional Testing

Through perpetually rotating gears and relentlessly moving clock hands in this mechanical space, players experience the suffocating pressure of time and machinery in this steel jungle, feeling the alienation and loss of humanity under modern industrial civilization.

Inspiration: Chaplin's Modern Times
Performance: People are caught up in a huge gear mechanism
Moral: The devouring of humanity by industrialization, the alienation of individuals on the assembly line
AI random move

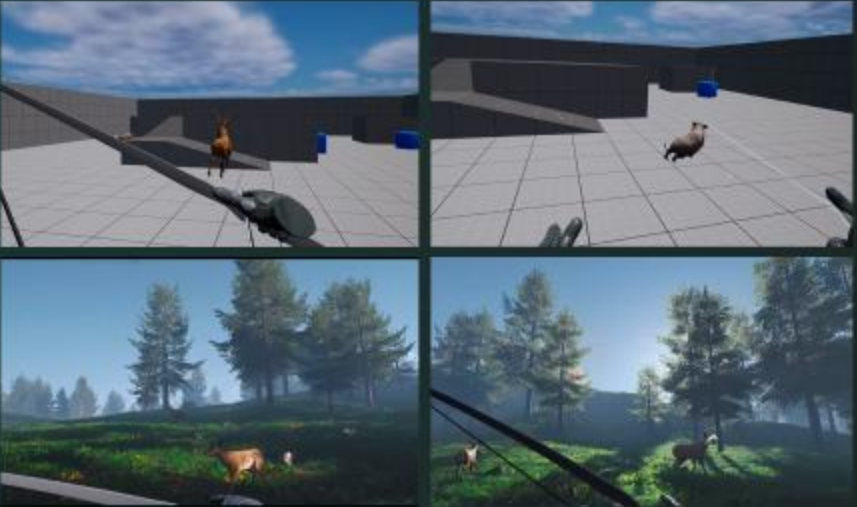
Archery Test



AI Mobile Module



Physical effects



High-quality rendering



I have enhanced the VR archery experience through high quality rendering and physics effects. The flight path of the arrow, target reaction and object collision are all realistically simulated by the physics engine, and the light and shadow effects and environment interactions make the whole scene more vivid and immersive.